

Úvod do paralelního programování

1. Semestrální práce

Analýza meteorologických dat

Jméno: Jan Kopačka
Osobní číslo: A24B0346P
Email: jankop@students.zcu.cz
Datum: 8. května 2026

Obsah

1	Úvod	3
2	Vývojové a testovací prostředí	4
2.1	Softwarové prostředí	4
2.2	Hardwarové vybavení	4
3	Analýza úlohy a implementace	5
3.1	Rozdělení modulů	5
3.1.1	Modul vstupu (data_loader.h, data_loader.cpp)	5
3.1.2	Modul zpracování (data_processor.h, data_processor.cpp)	5
3.1.3	Modul výstupu (output_generator.h, output_generator.cpp, cmap_svg.h)	5
3.1.4	Společná infrastruktura (data_types.h, main.cpp)	5
3.2	Paralelizace	6
3.2.1	Precedenční graf	6
3.2.2	Kritická cesta a průměrná míra paralelismu	7
3.2.3	Identifikace vhodných míst pro paralelizaci	7
3.3	Implementace	8
3.3.1	Stručný popis	8
3.3.2	Další učiněná rozhodnutí a vysvětlení implementačních detailů	8
3.3.3	Naměřené hodnoty	10
4	Metriky relativní k paralelizaci	11
4.1	Urychlení	11
4.2	Efektivita paralelizace	11
4.3	Amdahlův zákon	12
4.4	Gustafsonův zákon	12
4.5	Karpova-Flattova metrika	13
5	Zhodnocení implementace a závěr	14

Kapitola 1

Úvod

Cílem této semestrální práce je návrh a implementace vysoce výkonné aplikace pro analýzu a vizualizaci meteorologických dat z území České republiky. Práce vychází z reálných měření poskytnutých Českým hydrometeorologickým ústavem (ČHMÚ), která byla pro účely testování škálovatelnosti rozšířena do tří úrovní obtížnosti (malá, střední a velká datová sada).

Hlavním úkolem aplikace je zpracování dvou typů vstupních dat v textovém formátu CSV: seznamu měřicích stanic a souboru obsahujícího průměrné denní teploty. Zpracování zahrnuje několik klíčových fází:

- **Předzpracování a filtrace:** Identifikace stanic s dostatečnou kvalitou dat, definovanou jako přítomnost alespoň 5 let souvislých měření a průměrná hustota záznamů vyšší než 100 hodnot ročně.
- **Detekce anomálií:** Vyhledávání meziročních výkyvů teplot v jednotlivých měsících na základě statistického prahu (75 % celkového rozpětí teplot dané stanice v daném měsíci).
- **Vizualizace:** Generování dvanácti map České republiky ve formátu SVG (jedna pro každý kalendářní měsíc), kde jsou stanice barevně interpolovány na škále modrá–žlutá–červená podle průměrných teplot.

Z technického hlediska je kladen důraz na efektivitu a paralelizaci výpočtů. Aplikace je implementována v jazyce C++ s využitím knihovny **OpenMP** pro distribuci zátěže mezi více jader procesoru. Program umožňuje běh v sériovém i paralelním režimu, což slouží k následné analýze výkonu, výpočtu reálného urychlení a ověření teoretických předpokladů na základě Amdahlova a Gustafsonova zákona.

Dokumentace v následujících kapitolách podrobně popisuje zvolenou architekturu, proces identifikace úseků vhodných pro paralelizaci a vyhodnocení naměřených výsledků na testovacím hardwaru.

Kapitola 2

Vývojové a testovací prostředí

2.1 Softwarové prostředí

Program byl vyvíjen na operačním systému Windows 11 Pro. Jako hlavní vývojové prostředí (IDE) bylo využito CLion od společnosti JetBrains, přičemž o překlad se staral kompilátor g++ v prostředí MinGW. Pro ověření kompatibility a správného fungování byl kód rovněž odzkoušen v prostředí Microsoft Visual Studio 2022 s využitím překladače MSVC.

2.2 Hardwarové vybavení

Vývoj aplikace a měření prezentovaných výsledků probíhalo na počítačové sestavě s následující specifikací:

- **Procesor (CPU):** AMD Ryzen 5 7600X (6 fyzických jader, 12 vláken)
- **Operační paměť (RAM):** 32 GB DDR5 (6000 MHz, CL36)
- **Grafická karta (GPU):** NVIDIA GeForce RTX 4070 (12 GB GDDR6X)
- **Úložiště:** 1 TB NVMe SSD (Kingston KC3000)

Kapitola 3

Analýza úlohy a implementace

3.1 Rozdělení modulů

Pro dosažení čistoty kódu byly jednotlivé úlohy seskupeny do několika modulů podle jejich charakteru. Seznam úloh je vizuálně popsán v kapitole Precedenční graf.

3.1.1 Modul vstupu (`data_loader.h`, `data_loader.cpp`)

Tento modul zapouzdřuje tasky, které pracují s externími zdroji dat a transformují je do vnitřních struktur aplikace.

- **Načtení stanic** – Prvotní inicializace mapy stanic ze souboru `stanice.csv`.
- **Načtení measurements** – Naplnění existujících struktur daty ze souboru `mereni.csv`.

3.1.2 Modul zpracování (`data_processor.h`, `data_processor.cpp`)

Zde probíhá výpočetně nejnáročnější zpracování dat. Modul obsahuje úlohy, které mění stav datové sady nebo z ní extrahují nové informace.

- **Filtrace stanic** – Odstranění stanic nesplňujících podmínku 5 let souvislých dat a průměrného počtu měření. Součástí je i předvýpočet statistik.
- **Detekce anomálií** – Algoritmus pro identifikaci meziročních teplotních výkyvů na základě 75% prahu.

3.1.3 Modul výstupu (`output_generator.h`, `output_generator.cpp`, `czmap_svg.h`)

Tento modul se stará o tasky spojené s prezentací výsledků.

- **Export anomálií** – Zápis nalezených výkyvů do souboru `vykyvy.csv`.
- **Export SVG** – Generování 12 map s lineární interpolací souřadnic a barevnou škálou.

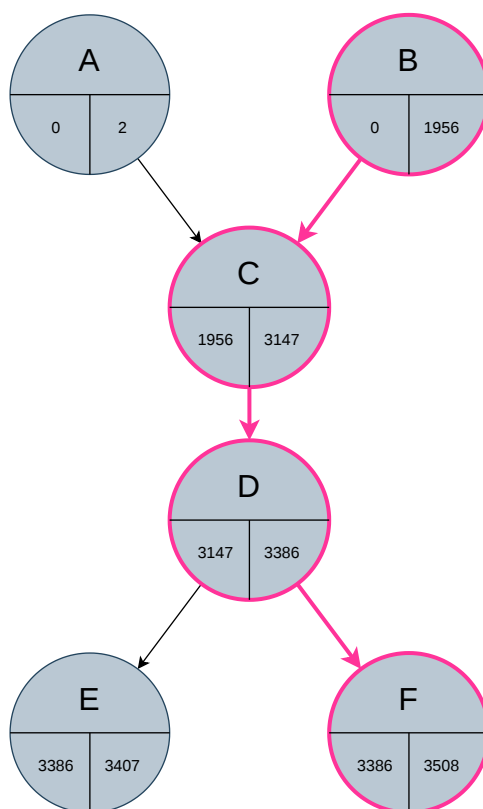
3.1.4 Společná infrastruktura (`data_types.h`, `main.cpp`)

- **`data_types.h`**: Obsahuje definice struktur `Station`, `Measurement` a `Anomaly`. Tyto struktury představují pevně definované datové rozhraní mezi všemi moduly.
- **`main.cpp`**: Funguje jako hlavní řídicí bod, který tasky spouští v pořadí definovaném v precedenčním grafu.

3.2 Paralelizace

3.2.1 Precedenční graf

Časy v precedenčním grafu a tabulce jsou ze sériové varianty na velkých datech. Hodnoty jsou skutečně naměřené, avšak v tomto kontextu slouží spíše jako referenční ukázka a kontext k precedenčnímu grafu.



Obrázek 3.1: Precedenční graf se znázorněnou kritickou cestou

Označení	Popis	Pr. čas (ms)
A	Načtení stanic z CSV souboru	2
B	Načtení měření jednotlivých stanic z CSV souboru	1901
C	Filtrace stanic na základě jejich měření	1173
D	Detekce anomálií stanic, které byly zachovány	244
E	Export CSV souboru anomálií	21
F	Export 12 SVG map pro každý měsíc	127

Tabulka 3.1: úlohy a naměřené časy sériové varianty k precedenčnímu grafu

3.2.2 Kritická cesta a průměrná míra paralelismu

Nejprve stanovíme celkový objem práce T (Total Work), který odpovídá součtu časů všech podúloh v sériovém režimu:

$$T = A + B + C + D + E + F = 2 + 1901 + 1173 + 244 + 21 + 127 = 3468 \text{ ms} \quad (3.1)$$

Dále identifikujeme délku kritické cesty L (Critical Path), představující nejdelší možnou posloupnost na sobě závislých úloh. V našem případě vede tato cesta přes uzly $B \rightarrow C \rightarrow D \rightarrow F$:

$$L = 1901 + 1173 + 244 + 127 = 3445 \text{ ms} \quad (3.2)$$

Z těchto hodnot lze vypočítat průměrnou míru paralelismu d , která udává teoretické urychlení v případě paralelizace výhradně na úrovni úloh (Task Parallelism):

$$d = \frac{T}{L} = \frac{3468}{3445} \approx 1,007 \quad (3.3)$$

Tato hodnota blízká 1 potvrzuje, že závislosti mezi úlohami jsou natolik silné, že souběžné spouštění celých tasků (např. pomocí `omp sections`) nepřináší významný výkonnostní zisk.

3.2.3 Identifikace vhodných míst pro paralelizaci

Fáze načítání a parsování dat

V této fázi představuje hlavní limitaci sekvenční charakter vstupu z disku (I/O). Nicméně samotný převod textových řetězců na číselné datové typy a jejich ukládání do vnitřních struktur je operace výpočetně náročná a lze ji paralelizovat. Vhodným přístupem se jeví rozdělení paměťového bufferu na několik nezávislých bloků (chunks). Pokud budou hranice bloků dynamicky zarovnány na konec řádku, mohou jednotlivá vlákna zpracovávat své úseky zcela izolovaně. Potenciální kolize při zápisu do sdílené hashmapy lze řešit buď využitím lokálních úložišť pro každé vlákno, nebo synchronizovaným vkládáním v závěrečné fázi načítání.

Fáze filtrace stanic

Proces filtrace je ideálním kandidátem na paralelizaci díky své povaze (tzv. embarrassingly parallel problem). Jelikož validace podmínek (kontrola 5 let souvislých měření a frekvence dat) probíhá pro každou stanici zcela nezávisle na ostatních, je možné rozdělit vektor stanic mezi dostupná vlákna. Jako efektivní strategie se zde jeví využití pomocné masky (bitsetu či vektoru příznaků), kam budou vlákna zaznamenávat výsledek analýzy. Tím se předejde nutnosti zamykání sdíleného vektoru při odstraňování prvků, což by degradovalo výkon.

Fáze detekce anomálií

Výpočet meziročních výkyvů vyžaduje komplexní statistické vyhodnocení (hledání extrémů a výpočet průměrů) pro každou stanici a měsíc. Vzhledem k tomu, že data jedné stanice nijak neovlivňují analýzu stanice druhé, lze výpočetní zátěž rovnoměrně rozprostřít. Paralelní cyklus může obsluhovat vnější smyčku přes seznam stanic. Protože se očekává, že počet detekovaných anomálií bude řádově nižší než celkový počet měření, bude dopad synchronizace při vkládání výsledků do finálního seznamu minimální.

Fáze generování výstupů

Vizualizace dat nabízí dvě úrovně paralelismu. První je nalezení globálních extrémů (min/max) pro kalibraci barevné škály, kde lze efektivně uplatnit paralelní redukci napříč všemi naměřenými hodnotami. Druhou úrovní je samotné generování 12 SVG map. Jelikož je každý měsíc reprezentován samostatným souborem, lze proces rozdělit na 12 nezávislých úloh. Každé vlákno tak může obsluhovat vlastní souborový proud (file stream), čímž se zcela eliminuje potřeba jakékoliv synchronizace při zápisu na disk.

3.3 Implementace

3.3.1 Stručný popis

Pro paralelizaci výpočtů byla zvolena knihovna **OpenMP**. Hlavním důvodem této volby je zachování vysoké přehlednosti a čistoty zdrojového kódu. Paralelizace je realizována pomocí direktiv kompilátoru (`#pragma omp`), které umožňují transformovat kritické úseky algoritmu na paralelní bez nutnosti rozsáhlého refaktoringu nebo implementace komplexní správy vláken. Tento přístup zajišťuje, že kód zůstává srozumitelný a blízký své sekvenční logice.

Program disponuje mechanismem pro přepínání mezi sekvenčním a paralelním vykonáváním na bázi příznaku `is_parallel`. Tento logický parametr je inicializován v modulu `main.cpp` analýzou argumentů příkazové řádky (`--serial` nebo `--parallel`). Hodnota příznaku je následně distribuována do všech klíčových metod loaderu, procesoru i generátoru. Většina paralelních bloků v kódu využívá podmíněnou klauzuli `if(is_parallel)`, což zaručuje identické chování algoritmu v obou režimech a eliminuje režii OpenMP v případech, kdy je vyžadován sériový běh.

Tok dat mezi jednotlivými moduly je navržen pro maximální efektivitu a nízkou paměťovou režii:

1. **Načítání (Modul vstup):** Data jsou parsována z CSV souborů. Pro úvodní přiřazení měření ke stanicím je využita `std::unordered_map` pro dosažení konstantní časové složitosti vyhledávání podle ID stanice.
2. **Transformace:** Jakmile jsou všechna data v paměti, hash mapa je převedena na `std::vector`. Tento krok je klíčový pro následnou paralelizaci, protože souvislé uložení v paměti umožňuje optimální využití CPU cache a snadné rozdělení iteračního prostoru mezi vlákna.
3. **Zpracování (Modul zpracování):** Nad vektorem stanic probíhá filtrace (odstranění stanic nesplňujících kritéria) a výpočetní analýza pro detekci anomálií. Výsledkem je vektor struktur `Anomaly`.
4. **Výstup (Modul výstup):** Data o anomáliích jsou sekvenčně zapsána do CSV souboru. Nicméně byl pokus o současně paralelní generování 12 SVG map, kde každé vlákno nezávisle konstruuje grafickou reprezentaci pro konkrétní měsíc v roce. **Tento postup se ale projevil jako neefektivní**, kde kvůli zvýšené režii, došlo k necelému dvojnásobnému zhoršení výkonu. Více podrobností o tomto problému v sekci 3.3.2.

Výše uvedený text slouží jako stručný přehled architektury a logiky programu. Pro detailní pochopení implementace jednotlivých metod a datových struktur je doporučeno nahlédnout přímo do zdrojového kódu, kde jsou zejména hlavičkové soubory opatřeny podrobnými dokumentačními komentáři v anglickém jazyce.

3.3.2 Další učiněná rozhodnutí a vysvětlení implementačních detailů

Filtrace stanic

Hlavním cílem této sekce je upřesnit způsob filtrování stanic. Důvodem je to, že v zadání je v rámci této části drobný prostor pro vlastní interpretaci.

Základním kritériem je počet let, přičemž pokud má stanice záznamy z méně než 5 různých let, je z dalšího zpracování vyřazena. Toto je nejrychlejší filtr.

Důležitá je také souvislost let. Pokud stanice měřila například v letech 1990–1993 (4 roky) a pak až v letech 2000–2003 (další 4 roky), má sice 8 let dat, ale neprojde, protože její nejdelší souvislý blok je pouze 4 roky dlouhý. Pro úspěšnou filtraci musí existovat alespoň jeden blok o délce minimálně 5 let.

Dalším požadavkem je dostatečná frekvence měření. Pokud stanice měřila například v období let 2000 až 2010 (období 11 let), musí mít v součtu alespoň 1100 záznamů, aby byla považována za dostatečně aktivní. Toto pravidlo platí i v případě, kdy stanice měřila pouze v roce 2000 a

posléze až v roce 2010. Situace se interpretuje jako období od 2000 do 2010 (11 let), ačkoliv reálně probíhalo měření jen dva roky, výpočet požadované frekvence je identický a je třeba dosáhnout 1100 záznamů.

Práce s I/O operacemi

Vstupní soubory jsou otevírány v binárním režimu pomocí příznaku `std::ios::binary`. Tento krok primárně zrychluje proces čtení, jelikož operační systém nemusí provádět skrytou textovou konverzi konců řádků (např. `\r\n` na `\n` ve Windows). Dává nám to absolutní kontrolu nad parsováním formátu a správným rozdělením znaků, přičemž celý soubor je načten naráz do alokovaného paměťového bufferu, abychom minimalizovali I/O operace.

Paralelizace čtení je umožněna tím, že je soubor načten do jednoho velkého řetězce, který program následně rozdělí na bloky (chunks) odpovídající počtu dostupných vláken. Hranice každého bloku se dynamicky posunou na nejbližší znak konce řádku (`\n`), aby se předešlo přeseknutí záznamu v polovině. Každé vlákno pak nezávisle parsuje pouze svůj vymezený paměťový blok.

Samotný zápis výsledných anomálií do CSV souboru byl ponechán jako sekvenční operace. Snaha o paralelizaci zápisu do jednoho sdíleného souboru by vyžadovala striktní synchronizaci pomocí kritických sekcí nebo zámků, jejíž režie by při předpokládaném objemu výsledků zcela eliminovala jakékoliv zrychlení.

Podobný problém byl i u pokusu o paralelizaci generování 12 map měsíců. Zde v rámci testování, bylo zjištěno, že při pokusu generování 12 souborů současně se naměřený čas, této konkrétní sekce zvýšil z 127 ms na 210 ms. Což je zhruba dvakrát horší. Hlavním důvodem je režie souběžného otevírání více souborových proudů a dosažení fyzických limitů propustnosti diskového úložiště (I/O-bound limit). Ačkoliv každé vlákno obsluhuje vlastní souborový proud bez nutnosti synchronizace v kódu, paralelní zápis na disk způsobuje fragmentaci přístupu k médiu. Režie operačního systému na správu těchto souběžných operací a přepínání kontextu u I/O úloh v tomto případě zcela eliminovala a následně převýšila časový zisk z paralelního výpočtu dat v paměti

Pro dosažení maximální propustnosti dat probíhá parsování číselných hodnot z bufferu zcela manuálně pomocí přímé manipulace s ASCII znaky (např. výpočtem `id = id * 10 + (buffer[pos] - '0')`), čímž se zcela vyhýbáme volání standardních funkcí jako `std::stoi` nebo `std::atoi`. Tento přístup eliminuje režii spojenou s alokací pomocných řetězců, kontrolou lokalizace a vnitřní stavovou logikou standardních proudů, která je při zpracování milionů řádků kritickým úzkým hrdlem. V kombinaci s předem načteným bufferem tato nízkoúrovňová optimalizace umožňuje dosáhnout řádově vyššího výkonu, což je pro efektivní využití paralelních vláken u datově intenzivních úloh klíčové.

Volba datových struktur

Při volbě datových struktur je pro počáteční načtení měření využita `std::unordered_map` (hash mapa). Hlavním důvodem je nutnost okamžitého vyhledávání stanice dle jejího ID pro přiřazení naměřené hodnoty se složitostí $O(1)$. V polovině procesu, těsně před započítáním analytických výpočtů, je však obsah hash mapy přesunut do souvislého paměťového bloku `std::vector`. Vektory lépe využívají paměť typu CPU cache a především umožňují efektivní a korektní rozdělení práce v blocích při použití `#pragma omp parallel for`.

Aby byla zajištěna férovost měření, probíhá tento paměťový převod zcela shodně jak v paralelní, tak v sériové variantě běhu programu. Smyslem je zajistit maximální jednotnost algoritmické logiky a paměťové stopy. Pouze v případě, že sériová verze provádí identickou režii, jsou získané metriky zrychlení spravedlivé a skutečně vypovídají o čistém výkonnostním zisku ze zapojení více vláken do výpočtů.

Specifika využití knihovny OpenMP

Z hlediska datových typů iterátorů mohl být díky využití standardu OpenMP 3.0+ použit pro iterační proměnné v paralelních for cyklech typ `size_t` namísto tradičních znaménkových celých

čísel (`int`). Při iteraci přes rozsáhlé kolekce dat, jakou je například `std::vector`, představuje `size_t` nativní a mnohem bezpečnější řešení, jež zamezuje potenciálnímu přetečení u extrémně velkých souborů.

Specifickým rozhodnutím bylo vyloučení vnořeného paralelismu a sekcí (`omp sections`). Předcedenční graf by teoreticky nabízel možnost spouštět některé odlišné tasky paralelně vedle sebe, například načítat stanice a měření ve stejný čas, nicméně tato metoda nebyla implementována. Knihovna OpenMP je totiž navržena tak, že každá úroveň paralelizace alokuje všechna smysluplná vlákna (např. 12 vláken pro CPU). Kdyby došlo ke spuštění více tasků paralelně nad sebou, systém by masivně překročil počet hardwarových vláken (`oversubscription`). Místo zrychlení by došlo k extrémnímu nárůstu režie na přepínání kontextů operačního systému a výkon by se naopak propadl. Z tohoto důvodu program paralelizuje data výhradně na úrovni dat uvnitř jednotlivých tasků.

Toto tvrzení bylo navíc potvrzeno v sekci 3.2.2, kde průměrná míra paralelismu vyšla velmi blízko hodnotě 1. To je jednoznačný ukazatel, že úlohy jsou na sobě silně závislé a paralelizace na úrovni úloh, by mohlo být kvůli vyšší režii spíše na škodu.

Zjednodušení distribuce šablony

Aby se minimalizovala chybovost a složitost nasazení aplikace u koncového uživatele (například špatně zadaná relativní cesta z příkazové řádky), byl vizuální základ slepé mapy (`<?xml version="1.0"... <svg>`) vložen přímo do hlavičkového souboru `czmap_svg.h` jako takzvaný surový řetězec (raw string literal `R"(...)"`). Aplikace tak nepotřebuje žádné externí zdrojové soubory a vystačí si (teoreticky) čistě se zkompilevanou binárnou.

3.3.3 Naměřené hodnoty

Datová sada	Sériová verze [ms]	Paralelní verze [ms]
Malá	262	111
Střední	1 116	439
Velká	3 570	1 331

Tabulka 3.2: Zpracování pro různé velikosti datových sad.

Kapitola 4

Metriky relativní k paralelizaci

4.1 Urychlení

Následující část se věnuje vyhodnocení efektivity paralelizace výhradně na velké datové sadě, kde se naplno projevuje vliv distribuce zátěže mezi jednotlivá jádra procesoru. V tabulce jsou uvedeny časy pro jednotlivé úlohy (tasky) definované v precedenčním grafu.

Úloha (Task)	Sériová varianta [ms]	Paralelní varianta [ms]
Načtení stanic (A)	2	2
Načtení měření (B)	1 901	888
Filtrace stanic (C)	1 173	187
Detekce anomálií (D)	244	19
Export anomálií (E)	21	20
Export SVG map (F)	127	120
Ostatní	1	4
Celkový čas	3 570	1 240

Tabulka 4.1: Srovnání časů jednotlivých úloh na velké datové sadě.

Na základě celkových naměřených časů lze vypočítat reálné urychlení na základě odezvy S_{lat} (speedup) aplikace. Výpočet je dán poměrem času sériového a paralelního běhu:

$$S_{lat} = \frac{T_{sériová}}{T_{paralelní}} = \frac{3570}{1240} \approx 2,88 \quad (4.1)$$

Dosažené reálné urychlení $S \approx 2,88$ na 12vláknovém procesoru ukazuje na měřitelné zrychlení výpočetně náročných částí (B, C, D). Výsledek potvrzuje, že pro rozsáhlé meteorologické soubory paralelní zpracování převážně pomáhá.

4.2 Efektivita paralelizace

Efektivita paralelizace η vyjadřuje míru využití dostupných hardwarových prostředků a je definována jako poměr dosaženého urychlení S a počtu procesorových jader (vláken) s . Ideálně se tato hodnota blíží k 1 (100 %), což by znamenalo ideální paralelizaci.

$$\eta = \frac{S}{s} = \frac{2,88}{12} \approx 0,24 \text{ (24 \%)} \quad (4.2)$$

Zhodnocení výsledků: Nízká efektivita (24 %) ukazuje, že přidání dalších vláken již nepřináší odpovídající zisk. Důvodem je povaha datově orientované úlohy. Zatímco čisté výpočetní části (jako detekce anomálií, kde čas klesl z 244 ms na 19 ms) se blíží ideálnímu škálování, operace závislé na propustnosti paměti a disku (čtení velkých CSV souborů, zápis do více SVG souborů) narážejí na hardwarové limity.

4.3 Amdahlův zákon

Amdahlův zákon předpovídá maximální teoretické urychlení aplikace při paralelizaci za předpokladu, že část algoritmu zůstává striktně sériová. Na základě dodatečného měření vnitřních částí modulu načítání bylo zjištěno, že čisté čtení souborů z disku do paměti a několik navazujících inicializačních operací v úlohách **A**, **B**, **F** tvoří fixní sériovou režii, kterou nelze paralelizovat ani v rámci paralelní varianty běhu. Jedná se o primárně I/O operace. Díky tomu máme kompletní obrázek o tom, jak moc přibližně je velká fixní sériová část. Jedná se o části úlohy dříve zmíněných úloh a celou úlohu **E**, která v implementaci paralelizována není. Konkrétní časy těchto sériových částí jsou:

- u úlohy A (Načtení stanic): 0,2 ms
- u úlohy B (Načtení měření): 349 ms
- u úlohy F (Zápis 12 SVG souborů): 84 ms

Vzorec pro výpočet teoretického maxima S_{max} je:

$$S_{max} = \frac{1}{(1-p) + \frac{p}{s}} \quad (4.3)$$

Kde p je podíl paralelizované části a $(1-p)$ podíl sériové části. Na základě naměřených dat pro velkou sadu (Tabulka 4.1) a započtení vnitřních sériových úseků loaderu určíme celkovou dobu sériových operací T_S takto:

$$T_S = T_E + T_{A_serial} + T_{B_serial} + T_{F_serial} = 21 + 0,2 + 349 + 84 = 454,2 \text{ ms} \quad (4.4)$$

Podíl sériové části $a = (1-p) = \frac{454,2}{3570} \approx 0,127$ (12,7 %). Podíl paralelizovatelné části $p = 1 - 0,127 = 0,873$ (87,3 %). Pro náš 12vláknový procesor je teoretické urychlení:

$$S_{max_12} = \frac{1}{0,127 + \frac{0,873}{12}} = \frac{1}{0,127 + 0,0728} \approx 5,01 \quad (4.5)$$

Zhodnocení výsledků: Dosazením do modelu Amdahlova zákona získáme teoretickou horní hranici urychlení ve výši $5,01 \times$. Skutečnost, že reálně dosažené urychlení celého programu ($2,68 \times$) je o 46,5 nižší než toto teoretické maximum, je způsobena kombinací limitované propustnosti paměťové sběrnice (memory-bound), režie knihovny OpenMP na správu vláken, nutné serializace v kritických sekcích a snížení taktovací frekvence procesoru při plném vytížení všech jader.

4.4 Gustafsonův zákon

Gustafsonův zákon představuje alternativní pohled na urychlení paralelního výpočtu. Na rozdíl od Amdahlova zákona předpokládá, že s rostoucím počtem procesorů se úměrně zvyšuje i objem zpracovávaných dat (workload), zatímco čas provádění zůstává konstantní. Tento přístup lépe reflektuje reálné nasazení vysoce výkonných aplikací na velkých datových sadách. Vzorec pro výpočet urychlení S je definován jako:

$$S = (1-a)P + a \quad (4.6)$$

Kde a je podíl sériové části kódu a P je počet procesorových jader. S využitím dříve stanoveného podílu sériové části $a = 0,127$ a počtu vláken $P = 12$ získáme:

$$S = (1 - 0,127) \cdot 12 + 0,127 = 0,873 \cdot 12 + 0,127 = 10,476 + 0,127 \approx 10,60 \quad (4.7)$$

Zhodnocení výsledků: Gustafsonův zákon predikuje teoretické urychlení $10,60\times$. Tato hodnota je výrazně vyšší než u Amdahlůva modelu ($5,01\times$), protože Gustafsonův zákon předpokládá škálování velikosti problému spolu s výpočetní kapacitou. Rozdíl mezi touto hodnotou a reálným urychlením ($2,88\times$) opět poukazuje na vliv fyzických limitů hardwaru a režie synchronizace, které nejsou v tomto lineárním modelu zahrnuty.

4.5 Karpova-Flattova metrika

Karpova-Flattova metrika slouží k odhadu reálného podílu sériového kódu v paralelním programu na základě naměřeného urychlení. Tato metrika doplňuje Amdahlův a Gustafsonův zákon tím, že kvantifikuje efektivitu paralelizace vzhledem k počtu jader.

Odhad podílu sériové části e se vypočte podle vztahu:

$$e = \frac{\frac{1}{\psi} - \frac{1}{p}}{1 - \frac{1}{p}} \quad (4.8)$$

Kde ψ je reálné urychlení programu a p je počet procesorových jader. Dosazením naměřených hodnot ($\psi = 2,88$; $p = 12$) získáme:

$$e = \frac{\frac{1}{2,88} - \frac{1}{12}}{1 - \frac{1}{12}} = \frac{0,3472 - 0,0833}{1 - 0,0833} = \frac{0,2639}{0,9167} \approx 0,288 \text{ (28,8 \%)} \quad (4.9)$$

Zhodnocení výsledků: Vypočtená hodnota $e \approx 0,288$ indikuje, že reálný podíl sériové složky výpočtu (včetně veškeré režie) činí necelých 29 %. Čím nižší je hodnota této metriky, tím kvalitnější je paralelizace algoritmu. Výrazný nárůst oproti původně naměřené fixní sériové režii (12,7 %) potvrzuje existenci významné skryté režie spojené se správou vláken a limitovanou propustností systémových prostředků při plném paralelním zatížení.

Kapitola 5

Zhodnocení implementace a závěr

Navržená aplikace pro analýzu meteorologických dat úspěšně demonstruje praktické využití paralelního programování pomocí knihovny OpenMP v jazyce C++. Implementace potvrdila, že pro dosažení vysokého výkonu u datově intenzivních úloh je nezbytná kombinace promyšlené paralelní strategie a nízkoúrovňových optimalizací správy dat. Klíčovým faktorem pro celkovou efektivitu programu bylo několik strategických rozhodnutí:

- **Optimalizace I/O operací a parsování:** Nahrazení standardních funkcí manuální manipulací s ASCII znaky přímo v načteném paměťovém bufferu eliminovalo obrovskou režii spojenou s alokacemi paměti a lokalizací.
- **Transformace datových struktur:** Využití počáteční vyhledávací složitosti hash mapy a její následný převod do souvislého vektoru zajistilo optimální využití CPU cache a efektivní, bezpečné rozdělení iteračního prostoru mezi jednotlivá vlákna.
- **Reflexe hardwarových limitů:** Měření jasně prokázala, že ne vše dává smysl paralelizovat. Pokus o souběžný zápis dvanácti SVG map odhalil fyzické limity propustnosti diskového úložiště. Fragmentace přístupu k médiu výkon natolik degradovala, že ponechání této části v čistě sériovém režimu bylo jediným racionálním krokem.
- **Omezení přesycení vláken:** Vyloučení vnořené paralelismu a paralelních sekcí (omp sections) zabránilo extrémnímu nárůstu režie na přepínání kontextu operačního systému (oversubscription). Toto rozhodnutí navíc plně korespondovalo s analýzou kritické cesty, kde se průměrná míra paralelismu blížila hodnotě 1.

Naměřené výsledky na velké datové sadě vykazaly reálné urychlení zhruba $2,88\times$ na 12vláknovém procesoru s celkovou efektivitou paralelizace 24 %. Ačkoliv teoretické modely (Amdahlův a Gustafsonův zákon) predikovaly výrazně vyšší potenciál zrychlení, Karpova-Flattova metrika přesně identifikovala podstatu chování systému. Reálný podíl sériové složky výpočtu, který v sobě ukrývá i skrytou režii knihovny OpenMP a úzké hrdlo paměťové sběrnice, činil téměř 29 %. Výkon aplikace tedy narážel primárně na memory-bound a I/O-bound limity, nikoliv na nedostatek výpočetní síly samotných procesorových jader.

Závěrem lze konstatovat, že vytvořený program je plně funkční, stabilní a přenositelný mezi kompilátory. Implementace beze zbytku splnila stanovené cíle a poskytuje výkonný nástroj pro analýzu rozsáhlých datových sad. Práce tak úspěšně demonstruje nejen výhody paralelního zpracování, ale i jeho reálné limity v úlohách, kde dominuje objem a přesun dat nad jejich samotným výpočtem.